

File

常用的 File 方法

- `file.exists()`
- `file.length()>0`
- `file.getTotalSpace()`: 返回long类型
- `file.getFreeSpace()`
- `file.getUsableSpace()`
- `File.separator`: 等价于 “/” , 不同的系统分隔符不同

文件权限

Linux/Unix 中的文件权限模型

files	2015-03-28 08:02	drwxrwxr-x	
private.txt	7 2015-03-28 07:38	-rw-----	110 000 000 --> 600
public.txt	12 2015-03-28 07:55	-rw-rw-rw-	110 110 110 --> 666
readonly.txt	8 2015-03-28 08:02	-r--r--r--	110 110 100 --> 664
wirteonly.txt	9 2015-03-28 08:02	-rw-rw-rw-	110 110 010 --> 662

rw- --- --- 私有
rw- rw- rw- 公开
rw- rw- r-- 只读
rw- rw- -w- 只可写

chmod : change mode 更改文件访问的模式.

r W X r W X r W X

所有者读	所有者写	所有者执行	组读	组写	组执行	其他用户读	其他用户写	其他用户执行
------	------	-------	----	----	-----	-------	-------	--------

所有者

组

其他用户

文件权限模型 中的 权限位

- r: 可读
- w: 可写
- x: 可执行
- s:

作用于目录时，表示这是一个 "**粘滞位目录**"：只有文件的所有者或者超级用户才能删除/重命名该目录下的文件

作用于文件时，表示这是一个 "**同步写入**"：对文件的修改（文件内容、文件元数据），都会同步（s）到底层存储设备（磁盘）

权限位共分为 3 组：所有者/组/其他用户

安卓系统中的文件权限

1. 应用私有目录（/data/data/包名/）下创建的文件，默认都是私有的（2种特殊情况，可以公有）
 - 利用 `chmod` 更改文件权限
 - 配置清单文件的 `shareUserId`，可以使 2 个应用间的文件共享
2. 外置 SD 卡，通常都是公有的

更改文件权限

chmod 文件名 权限值 (600)

- 600: 私有 110 000 000
- 666: 共有 110 110 110

FileDescriptor

FileDescriptor **文件描述符**: 代表一个打开的文件或者 I/O 资源（它不包含文件路径和其他详细信息，而是代表一个指向实际资源的 **句柄**）

- 在 Linux/Unix 系统中，文件描述符是一个 非负整数
- 在安卓中，文件描述符是一个 FileDescriptor 对象

我们可以利用它进行一些低级别的 I/O 操作，如读写数据、套接字、管道或者其他支持流式传输的资源

代表一个 "操作系统级别的文件" 或 "套接字的描述符"，并提供了对文件的访问。

- **read()**、**write()**: 它们用于从文件中读取数据或将数据写入文件

这些方法可以在任何具有文件描述符的对象上调用，包括 FileInputStream、FileOutputStream、RandomAccessFile 等

- `getChannel()` 方法返回与该文件描述符关联的 `FileChannel` 对象
- `sync()` 方法将所有挂起的文件系统更改刷新到磁盘

创建

- 需要注意的是，`FileDescriptor` 类是不可继承的，因此不能创建新的实例。
- 相反，您可以使用 Android API 中提供的方法来获取现有的文件描述符。例如，您可以使用 `open()` 方法打开一个文件并获取与之关联的 `FileDescriptor` 对象。

总之，`FileDescriptor` 类是 Android 中用于访问文件和套接字的底层机制的一部分。它提供了一种在操作系统级别上操作文件和套接字的方式。

以下几个方面体现了 `FileDescriptor` 在 Android 中的使用：

1. 标准流：

每个 Android 进程中预定义了三个 `FileDescriptor` 实例，即 `System.in`、`System.out` 和 `System.err`，分别对应标准输入、标准输出和标准错误流。

2. 文件访问：

通过 `java.nio.channels.FileChannel` 或者 `java.io.FileInputStream`、`java.io.FileOutputStream` 等类可以关联到 `FileDescriptor`，进而对文件进行读写操作。

3. 跨进程共享：

在Android中，可以通过 `ParcelFileDescriptor` 将 `FileDescriptor` 序列化并跨进程传递，使得不同的应用程序之间能够安全地共享文件资源，如使用 `FileProvider` 来分享文件内容给其他APP

4. 网络与硬件交互：

对于网络套接字和设备文件（如摄像头、麦克风等），也可以通过 `FileDescriptor` 来实现与这些硬件资源的交互。

5. IPC机制：

在 Android 系统内部，`FileDescriptor` 还用于实现某些形式的进程间通信（IPC），比如在 `Looper` 机制中用于唤醒等待中的线程。

总之，Android 中的 `FileDescriptor` 是连接 Java 层 API 与底层操作系统之间的重要桥梁，它让开发者能够在不直接操作底层系统细节的情况下处理各种文件和 I/O 资源。

比较 2 个文件是否相同

比较 2 个文件是否相同

1. 逐个比较字节流
2. 比较文件的哈希值: DigestInputStream
3. 比较文件位置/地址: Files.isSameFile()

比较文件字节流

比较文件字节流: 读取两个文件的字节流, 逐个字节进行比较, 直到遇到不同的字节或文件结束

- 这种方法适合于比较较小的文件, 对于大文件效率较低

```
public static boolean compareFiles(String file1Path, String
file2Path) throws IOException {
    FileInputStream fis1 = new FileInputStream(file1Path);
    FileInputStream fis2 = new FileInputStream(file2Path);

    int byte1, byte2;
    do {
        byte1 = fis1.read();
        byte2 = fis2.read();
        if (byte1 != byte2) {
            return false;
        }
    } while (byte1 != -1 && byte2 != -1);
}
```

```
return byte1 == -1 && byte2 == -1;
}
```

比较文件哈希值

比较文件哈希值：使用 **MessageDigest + DigestInputStream** 计算文件的哈希值/消息摘要（如 MD5、SHA-1 等）

```
public static boolean compareFiles(String file1Path, String
file2Path) throws IOException, NoSuchAlgorithmException {

    // MessageDigest
    MessageDigest md = MessageDigest.getInstance("MD5");

    // DigestInputStream
    FileInputStream fis1 = new FileInputStream(file1Path);
    DigestInputStream dis1 = new DigestInputStream(fis1, md);

    while (dis1.read() != -1) ;

    // digest()
    byte[] file1Digest = md.digest();

    FileInputStream fis2 = new FileInputStream(file2Path);
    DigestInputStream dis2 = new DigestInputStream(fis2, md);
    while (dis2.read() != -1) ;
```

```
byte[] file2Digest = md.digest();

// MessageDigest.isEqual()
return MessageDigest.isEqual(file1Digest, file2Digest);
}
```

MessageDigest

MessageDigest：计算消息摘要/哈希值的类，提供了许多常见的哈希算法，如 MD5、SHA-1、SHA-256

1. 创建 `getInstance(String algorithm)`：算法名称常用的有 "MD5"、"SHA-1"、"SHA-256" 等
2. 更新 `update(byte[] input)`：使用给定的输入数据更新摘要
3. 计算 `digest()`：计算并返回摘要值，以字节数组形式表示
4. 比较 `isEqual()`
5. 重置 `reset()`：重置摘要值到初始状态，可以重新使用 MessageDigest 实例

```
public class DigestExample {

    public static byte[] calculateFileDigest(String filePath)
    throws IOException, NoSuchAlgorithmException {

        // MessageDigest
        MessageDigest md = MessageDigest.getInstance("MD5");
        FileInputStream fis = new FileInputStream(filePath);
```



```
DigestInputStream dis = new DigestInputStream(fis, md);

// 读取文件并更新摘要
byte[] buffer = new byte[8192];
while (dis.read(buffer) != -1) {
    // 读取文件的数据并更新摘要值
    md.update(buffer);
}

// 计算并返回摘要值
return md.digest();
}

public static void main(String[] args) {
    try {
        byte[] digest = calculateFileDigest("file.txt");
        System.out.println("File Digest: " +
bytesToHexString(digest));
    } catch (IOException | NoSuchAlgorithmException e) {
        e.printStackTrace();
    }
}

// 辅助方法：将字节数组转换为十六进制字符串
public static String bytesToHexString(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte b : bytes) {
        sb.append(String.format("%02x", b));           // 两位十六进制字
字符串
    }
    return sb.toString();
}
```

```
}
```

DigestInputStream

DigestInputStream: 一边读取数据, 一边计算数据的 "消息摘要/散列值"

```
public class DigestInputStreamExample {

    public static byte[] calculateFileDigest(String filePath)
    throws IOException, NoSuchAlgorithmException {
        MessageDigest md = MessageDigest.getInstance("MD5");
        InputStream is = new FileInputStream(filePath);
        DigestInputStream dis = new DigestInputStream(is, md);

        // 读取文件数据
        byte[] buffer = new byte[8192];
        while (dis.read(buffer) != -1) {
            // 读取数据, 同时计算摘要值
        }

        // 获取摘要值
        byte[] digest = md.digest();

        // 关闭流
        dis.close();
        is.close();

        return digest;
    }
}
```

```

public static void main(String[] args) {
    try {
        byte[] digest = calculateFileDigest("file.txt");
        System.out.println("File Digest: " +
bytesToHexString(digest));
    } catch (IOException | NoSuchAlgorithmException e) {
        e.printStackTrace();
    }
}

// 辅助方法：将字节数组转换为十六进制字符串
public static String bytesToHexString(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte b : bytes) {
        sb.append(String.format("%02x", b));
    }
    return sb.toString();
}
}

```

16 进制字符串

16 进制字符串：为什么要转 16 进制字符串？更方便地表示和输出二进制数据

1. 可读性：

十六进制字符串由 0–9 和 A–F 组成，更直观和易读。

相比于二进制形式，十六进制字符串更容易被人类理解和识别。

2. 兼容性：

十六进制字符串是一种常见的数据表示方法，广泛支持于各种编程语言和工具。

将摘要值转换为十六进制字符串后，可以方便地不同系统、平台和环境进行交互和处理。

3. 字符串表示：

摘要值通常需要通过文本形式进行存储、传输或展示。

将其转换为十六进制字符串后，可以作为字符串进行处理，比如存储到数据库、输出到文本文件等。

```
String.format("%02x", bytes[])
```

- **%**：表示占位符的起始。
- **0**：表示使用 0 填充。
- **2**：表示占位符的宽度为 2，不足 2 位时在前面补 0。
- **x**：表示将值格式化为十六进制。

Files.isSameFile()

Files.isSameFile()：是 Java NIO 中的一个静态方法

- 比较的是 2 个文件路径，是否指向同一个位置，而不是文件的

内容

- 如果两个文件的内容完全相同，如果它们位于不同位置或具有不同的文件名，则 `isSameFile()` 方法也会返回 `false`

```
public static boolean compareFiles(String file1Path, String
file2Path) throws IOException {
    Path path1 = Path.of(file1Path);
    Path path2 = Path.of(file2Path);

    return Files.isSameFile(path1, path2);
}
```